TP 6 : Algorithme de Cocke-Younger-Kasami

Préliminaires

Dans ce TP, on cherche à mettre en place un algorithme qui étant donnée une grammaire et un mot, teste si le mot est engendré par la grammaire.

On dit d'une grammaire est sous forme normale de Chomsky si toutes ses règles sont de la forme :

$$\begin{array}{ccc} A & \rightarrow & BC \\ A & \rightarrow & a \\ S & \rightarrow & \epsilon \end{array}$$

avec S le symbole initial, A, B et C des symboles non terminaux, a un symbole terminal et B et C différents de S.

On admet ici que tout grammaire non contextuelle peut être mise sous forme normale de Chomsky.

L'algorithme que l'on étudie aujourd'hui porte le nom de ces 3 inventeurs et est hors programme. Cependant, c'est un algorithme très classique qui pourra faire un excellent sujet de concours. De plus, il est relativement simple à comprendre.

Algorithme de Cocke-Younger-Kasami

L'algorithme de Cocke-Younger-Kasami est un algorithme de programmation dynamique. Si m est un mot de taille n, on lui associe le tableau à deux dimensions de taille $n \times n$. On cherche à remplir T pour vérifier la propriété suivante :

$$\forall i, j \in [\![0, n-1]\!], T[i][j] = \{X | X \Rightarrow^* m_i m_{i+1} \dots m_j\}$$

Moins formellement, $T[i][j]$ est l'ensemble des symboles non terminaux qui se dérivent en la portion de m de i à j. On remarque que $T[i][j]$ vérifie la propriété suivante :

- $X \in T[i][i]$ si la règle $X \rightarrow m_i$ est présente dans la grammaire.
- Si $i < j$, $X \in T[i][j]$ si et seulement si il existe $k \in [\![i, j-1]\!]$ et YZ tels que la règle $X \rightarrow YZ$ est présente dans G, $Y \in T[i][k]$ et $Z \in T[k+1][j]$. Moins formellement, s'il existe une décomposition du mot $m_i \dots m_j$ en deux mots engendré par Y et Z.

Ainsi, $S \in T[0][n-1]$ si et seulement si S se dérive en m.

On réalise ce TP en C. On étudiera les mots composés de caractères ASCII, soit approximativement, toutes les lettres de l'alphabet en minuscule et majuscule ainsi que les symboles non propres au français de votre clavier.

On considère la grammaire G suivante :

$$S \rightarrow XS \tag{1}$$
$$S \rightarrow Y \tag{2}$$
$$X \rightarrow YP \tag{3}$$
$$Y \rightarrow i, \quad \forall i \in [\![0, 9]\!] \tag{4}$$
$$P \rightarrow + \tag{5}$$

Question 1 : À quoi ressemble un mot de $L(G)$? Est-elle sous forme normale de Chomsky ? Si ce n'est pas déjà le cas, mettez là sous forme normale de Chomsky.

Question 2 : Exécuter l'algorithme CYK sur G et le mot $m = 1+2$.

Quelques types utiles

On utilisera les types suivants :

```
struct symbole
{
    bool est_epsilon;
    bool est_terminal;
    int identifiant;
    char valeur;
};
typedef struct symbole symbole;
```

Un symbole est composé d'un booléen indiquant s'il s'agit du mot vide, dun booléen indiquant s'il est terminal. Il dispose aussi d'un attribut indiquant son identifiant s'il est non terminal ou sa valeur s'il est terminal. On numérotera les symboles non terminaux.

```
struct regle
{
    int taille;
    symbole gauche;
    symbole* droite;
};
typedef struct regle regle;
```

On représente une règle par la taille de son membre de droite, son symbole de gauche ainsi qu'un tableau des symboles de droite.

```
struct chainon_regles
{
    regle valeur;
    struct chainon_regles* suivant;
};
typedef struct chainon_regles* liste_regles;

struct grammaire
{
    symbole initial;
    int nombre_non_terminaux;
    liste_regles regles_production;
};
typedef struct grammaire grammaire;
```

Pour représenter une grammaire, on utilise une liste d'adjacence des règles. On stocke aussi le nombre de non terminaux ainsi que le numéro du symbole initial.

```
struct tableau
{
    int nombre_non_terminaux;
    int taille_mot;
    bool*** derive_facteur;
};
typedef struct tableau tableau;
```

Cette structure permet de représenter T. La case `T[i][j]` contient un tableau de booléens. La case k de ce tableau indique si le k-ème symbole non terminal appartient à $T[i][j]$.

Question 3 : Écrire des fonctions `symbole creer_terminal(char c)` et `symbole creer_non_terminal(int n)` qui créent des symboles terminaux et non terminaux.

Question 4 : Écrire une fonction `void liberer_grammaire(grammaire g)` qui, étant donnée une grammaire, libère la mémoire allouée pour celle-ci.

Question 5 : Écrire une fonction `tableau initialiser_tableau(grammaire g, int taille_mot, char* mot)` qui, étant donnée une grammaire, initialise le tableau T associé.

Décomposition

Question 6 : Proposez une fonction `bool decompose(regle r, int i, int j, int k, char* mot, tableau T)` qui teste si une règle de la forme $X \to YZ$ permet d'affirmer $X \in T[i][j]$ pour un k donné.

Question 7 : Proposez une fonction `bool decomposable(regle r, int i, int j, char* mot, tableau T)` qui teste si une règle de la forme $X \to YZ$ permet d'affirmer $X \in T[i][j]$.

Question 8 : Proposez une fonction `void remplir(grammaire G, tableau T, char* mot)` qui remplit le tableau T à l'aide de l'ensemble des règles proposées.

Question 9 : Proposez une fonction `bool appartient(grammaire G, char* mot)` qui décide si `mot` appartient au langage engendré par `G`.

Question 10 : Tester votre programme. Je vous fournis le code suivant permettant de générer la grammaire donné en exemple.

```c
// Function to add a production rule to the grammar
void ajouter_regle(grammaire *g, regle r) {
    liste_regles new_chainon = malloc(sizeof(struct chainon_regles));
    new_chainon->valeur = r;
    new_chainon->suivant = g->regles_production;
    g->regles_production = new_chainon;
}

int main()
{
    grammaire g;
    g.nombre_non_terminaux = 5;
    g.initial = creer_non_terminal(0);
    g.regles_production = NULL;

    symbole S0 = creer_non_terminal(0);
    symbole S = creer_non_terminal(1);
    symbole X = creer_non_terminal(2);
    symbole Y = creer_non_terminal(3);
    symbole P = creer_non_terminal(4);

    symbole plus = creer_terminal('+');

    regle r;

    r.taille = 2;
    r.gauche = S0;
    r.droite = malloc(2 * sizeof(symbole));
    r.droite[0] = X;
    r.droite[1] = S;
    ajouter_regle(&g, r);

    for (char c = '0'; c <= '9'; c++) {
        r.taille = 1;
        r.gauche = S0;
        r.droite = malloc(1 * sizeof(symbole));
        r.droite[0] = creer_terminal(c);
        ajouter_regle(&g, r);
    }

    r.taille = 2;
    r.gauche = S;
    r.droite = malloc(2 * sizeof(symbole));
    r.droite[0] = X;
    r.droite[1] = S;
    ajouter_regle(&g, r);

    for (char c = '0'; c <= '9'; c++) {
        r.taille = 1;
        r.gauche = S;
        r.droite = malloc(1 * sizeof(symbole));
        r.droite[0] = creer_terminal(c);
        ajouter_regle(&g, r);
    }
```

```
    r.taille = 2;
    r.gauche = X;
    r.droite = malloc(2 * sizeof(symbole));
    r.droite[0] = Y;
    r.droite[1] = P;
    ajouter_regle(&g, r);

    for (char c = '0'; c <= '9'; c++) {
        r.taille = 1;
        r.gauche = Y;
        r.droite = malloc(1 * sizeof(symbole));
        r.droite[0] = creer_terminal(c);
        ajouter_regle(&g, r);
    }

    r.taille = 1;
    r.gauche = P;
    r.droite = malloc(1 * sizeof(symbole));
    r.droite[0] = plus;
    ajouter_regle(&g, r);

    liberer_grammaire(g);

    return 0;
}
```